evalci: A Python Library for Statistically Rigorous Comparison of Language Model Evaluations

Shreyas K Chandrahas

Abstract

The dominant practice in language model evaluation is to report a single accuracy number per model and declare the higher one better, without testing whether the gap could plausibly be sampling noise. On benchmarks of a few thousand items, and under temperature sampling where a model can differ from itself run to run by more than the reported gap between models, this practice routinely overstates confidence in headline claims. The statistical machinery to fix this – confidence intervals, paired significance tests, power analysis, clustered standard errors, multiple-comparison correction – is well established, but no standard, pip-installable tool packages it in the shape an evaluation actually takes: a per-item results table. We present `evalci`, a pure-Python library (numpy/scipy/pandas only) that turns a per-item results table into a publication-ready claim – e.g., "Model A beats Model B, $\Delta = 3.1$ pts, 95% CI [1.2, 5.0], paired permutation $p = 0.002$, $n = 1{,}319$" – in one function call, with adapters for `lm-evaluation-harness` and HELM output. Every routine is validated against an independent reference (`statsmodels`, or brute-force exact enumeration) rather than only against itself. As a case study, we re-analyze a public comparison of nine language models' MMLU accuracy and find that 3 of the 8 adjacent leaderboard-rank gaps are not statistically significant after correcting for the 36 pairwise comparisons the ranking implies. `evalci` is available at `https://pypi.org/project/evalci/` (source: `https://github.com/Shreyaskc/evalci`, DOI: `https://doi.org/10.5281/zenodo.21201815`).

1 Introduction

A language model evaluation is an experiment: a fixed set of items is scored against a metric, and the result is a noisy estimate of an underlying population quantity, not the quantity itself. The evaluation literature has largely treated it as something else – a deterministic measurement – and reported the resulting number without an error bar. This is not a cosmetic omission. On a benchmark of a few thousand items, a 2-point gap between two models' accuracy is frequently within the noise implied by the sample size alone. Under temperature sampling, the same model evaluated twice can differ from itself by more than the reported gap between two different models; recent work quantifying this shows that properly clustered standard errors – which account for repeated sampling of the same item – can be more than 3x larger than the naive standard errors typically reported [Miller, 2024].

This is not a new observation. Dror et al. [2018] surveyed significance testing practice in ACL and TACL papers and found it routinely ignored or misapplied. Card et al. [2020] showed that many standard NLP test sets are underpowered for the effect sizes papers actually claim. Miller [2024] derived the statistical machinery needed to analyze evaluations properly – clustered standard errors, power calculations – as a methods note. What has been missing is not the statistics but a standard, tested, pip-installable implementation shaped like an evaluation: a table of *items*, not a pair of scalars. `scipy` and `statsmodels` provide the general-purpose primitives (a proportion confidence

interval, a bootstrap, a multiple-testing correction), but none of them speak the vocabulary an evaluation actually produces – per-item, paired-by-question, clustered-by-repeated-decode, many-models-by-many-benchmarks – and none of them read `lm-evaluation-harness` or HELM output directly.

We present `evalci`, which closes that gap. Its contribution is not a new statistical method; every routine it implements is decades old. Its contribution is packaging: a small, tested, pure-Python library, installable with `pip` in under five minutes with no GPU, that takes the exact per-item table an evaluation harness already produces and returns a publication-ready confidence interval, significance test, or corrected many-model comparison table. Section 2 describes the library. Section 3 describes the statistical methods it implements. Section 4 describes how each was validated against an independent reference rather than only against itself. Section 5 uses `evalci` to re-analyze a public multi-model comparison and shows that a third of the adjacent leaderboard-rank gaps in it are not statistically significant once the comparisons actually being made are corrected for.

2 Library Design

`evalci` depends only on `numpy`, `scipy`, and `pandas` [Harris et al., 2020, Virtanen et al., 2020, McKinney, 2010], so `pip install evalci` works on a laptop with no GPU. The library is built around a single shared contract: a per-item results table with columns `item_id`, `model`, `score`, and optionally `subset` (for stratifying by benchmark or category) and `sample_idx` (for repeated decodes of the same item). Every function, and every adapter, produces or consumes this schema, so results loaded from an `lm-evaluation-harness` run, a HELM run, or a plain CSV compose freely.

The public API is six functions:

```
evalci.ci(scores, method="wilson"|"clopper-pearson"|"bootstrap")
evalci.compare(a, b, paired=True,
               method="permutation"|"bootstrap"|"mcnemar")
evalci.power(delta, n=None, power=0.8,
             method="analytic"|"simulation")
evalci.multi_compare(df, correction="holm"|"bh")
evalci.cluster_ci(scores, clusters)
evalci.report(result, format="markdown"|"latex")
```

A typical call looks like:

```
>>> result = evalci.compare(model_a_scores, model_b_scores,
... method="permutation")
>>> evalci.report(result)
'Δ=0.034, 95% CI [0.005, 0.063], paired permutation p=0.025*, n=1319'
```

Adapters (`evalci.adapters.load_lm_eval_harness`, `load_helm`, `load_csv`) parse per-item output from `lm-evaluation-harness` and HELM directly into the shared schema, and a CLI (`evalci compare results_a.json results_b.json`) wraps the common case of comparing two result files without writing any Python.

3 Statistical Methods

3.1 Confidence intervals

For a single model's binary (correct/incorrect) per-item scores, `evalci.ci` implements the Wilson score interval [Wilson, 1927] and the exact Clopper-Pearson interval [Clopper and Pearson, 1934].

For continuous scores, or when the binomial assumption is not appropriate, it wraps `scipy`'s bootstrap implementation [Virtanen et al., 2020] to provide percentile and bias-corrected-and-accelerated (BCa) intervals [Efron, 1979, Efron and Tibshirani, 1993].

3.2 Paired and unpaired comparison

`evalci.compare` implements three significance tests. The paired *sign-flip permutation test* randomly flips the sign of each per-item difference under the null that the two models are exchangeable on that item; for $n \leq 12$ items it enumerates all 2^n sign patterns exactly rather than approximating with Monte Carlo. The *null-shifted bootstrap hypothesis test* centers the observed differences at zero and resamples with replacement, giving a nonparametric alternative to the permutation test with the same asymptotic behavior. *McNemar's test* [McNemar, 1947], exact (binomial) below $n = 25$ discordant pairs and asymptotic (chi-squared, with continuity correction) above, tests paired binary outcomes directly via the discordant-pair counts. All three paired tests are accompanied by a bootstrap confidence interval on the mean paired difference, so a single `compare()` call returns both a significance test and an interval estimate. Unpaired variants (label-shuffle permutation, an independent-samples null-shifted bootstrap) are available when items are not aligned across the two models being compared – the situation in Section 5, where only aggregate accuracy, not per-item correctness, is public.

3.3 Multiple-comparison correction

`evalci.multi_compare` runs every pairwise model comparison (optionally stratified by benchmark subset) and applies Holm's step-down procedure [Holm, 1979] or Benjamini-Hochberg FDR control [Benjamini and Hochberg, 1995] across *all* of them – not just the subset a user might otherwise be tempted to report – before returning a table with adjusted p-values and a significance flag.

3.4 Clustered standard errors

`evalci.cluster_ci` resamples whole clusters, rather than individual observations, in its bootstrap – the correct treatment when items are not independent, as in repeated decodes of the same question under temperature sampling, or grouped items sharing an underlying passage. This directly addresses the clustering problem quantified by Miller [2024]: treating repeated samples as independent observations understates the true standard error.

3.5 Power analysis

`evalci.power` answers "how many items do I need to detect a δ-point gap at 80% power?" (or, given n, what power is achieved) via a closed-form two-proportion normal approximation, with a Monte Carlo simulation fallback (`method="simulation"`) that accepts a correlation parameter ρ – the fraction of items whose correctness is linked across the two models by shared difficulty rather than drawn independently – for when items are not independent across models. This is motivated directly by Card et al. [2020]'s finding that standard-sized NLP test sets are underpowered for many effect sizes papers report as significant.

4 Validation

Statistical correctness is the point of this library, so every routine is checked against an independent reference rather than only re-tested against its own implementation:

- Wilson and Clopper-Pearson intervals are checked against `statsmodels`'s `proportion_confint` [Seabold and Perktold, 2010] across a range of (k, n).
- McNemar's test, exact and asymptotic, is checked against `statsmodels`'s `mcnemar`.
- Holm and Benjamini-Hochberg correction are checked against `statsmodels`'s `multipletests`.
- The paired permutation test is checked against brute-force exact enumeration of all 2^n sign-flip patterns for small n, and Monte Carlo estimates are shown to converge to the exact value as the resample count grows.
- Bootstrap confidence intervals are checked with a coverage simulation: over 200 independent synthetic trials, a nominal 95% CI contains the true parameter in the expected range of trials.

`statsmodels` is a test-only dependency (installed via `pip install evalci[test]`), not a runtime dependency of the installed package – it is used only to validate `evalci`'s own, dependency-free implementations. At the time of writing, the test suite comprises 100 tests and covers 92–99% of the statistical core (`_intervals.py`, `_significance.py`, `_correction.py`, `_power.py`, `stats.py`); continuous integration runs the full suite on every push across Python 3.9–3.12.

5 Case Study: How Many Adjacent Leaderboard Ranks Are Actually Different?

As a concrete demonstration – and the kind of re-analysis `evalci` is meant to make routine – we ask how many of the rank differences implied by a public multi-model comparison table survive a significance test.

5.1 Data

We use Table 2 of Grattafiori et al. [2024], "Performance of finetuned Llama 3 models on key benchmark evaluations," which reports 5-shot MMLU accuracy for nine instruction-tuned models evaluated in a single table: Llama 3 8B, GPT-3.5 Turbo, Mixtral 8x22B, Nemotron-4 340B, Llama 3 70B, GPT-4 (0125), Llama 3 405B, GPT-4o, and Claude 3.5 Sonnet, with accuracies ranging from 69.4% to 89.9%. The MMLU test set contains $n = 14{,}042$ questions [Hendrycks et al., 2021].

5.2 Method

Per-item correctness for these models is not public – only the aggregate accuracy is. We reconstruct, for each model, an i.i.d. binary vector of length n consistent with its reported accuracy ($k =$ round(accuracy $\times\, n$) ones, the remainder zeros) and compare every pair with `multi_compare` using `correction="holm"`, `method="permutation"`, and `paired=False`. This is deliberately conservative: true per-item alignment, were it public, would let us run a *paired* test, and under the realistic assumption that item difficulty is positively correlated across models (some MMLU questions are simply harder for every model), a paired test is more powerful than the unpaired test used here. The count of non-significant gaps we report below is therefore an upper bound – a true paired re-analysis could only find *fewer* non-significant adjacent gaps, not more.

5.3 Result

`multi_compare` evaluates all $\binom{9}{2} = 36$ pairwise comparisons and Holm-corrects across all of them, not just the eight adjacent-rank pairs we highlight – the correct scope for the correction, since failing to account for comparisons actually performed (even ones not reported) is itself a common significance-testing error [Dror et al., 2018]. Figure 1 shows the resulting 95% Wilson confidence intervals for each model, with the three adjacent-rank pairs that are *not* significant at $\alpha = 0.05$ after correction shaded.

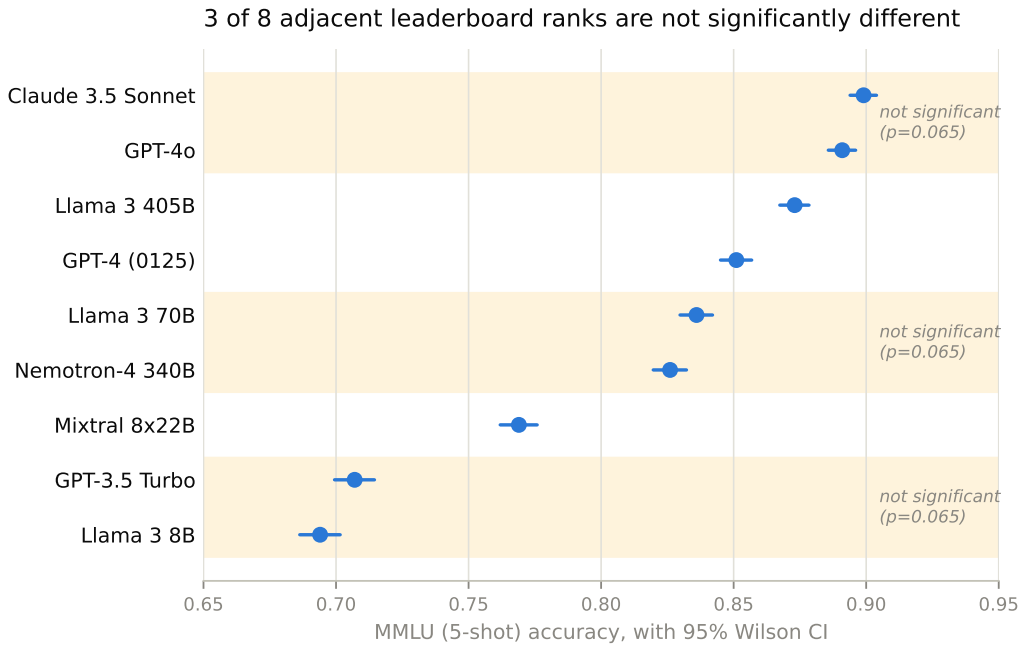

Figure 1: MMLU (5-shot) accuracy for nine publicly reported models, with 95% Wilson confidence intervals reconstructed from aggregate accuracy and $n = 14{,}042$. Shaded bands mark the three adjacent-rank pairs whose gap is not statistically significant ($p_{\text{adj}} > 0.05$) after Holm correction across all 36 pairwise comparisons.

Model A	Model B	Δ (pts)	p_{adj}	Significant?
GPT-3.5 Turbo	Llama 3 8B	1.3	0.065	No
Mixtral 8x22B	GPT-3.5 Turbo	6.2	0.0036	Yes
Nemotron-4 340B	Mixtral 8x22B	5.7	0.0036	Yes
Llama 3 70B	Nemotron-4 340B	1.0	0.065	No
GPT-4 (0125)	Llama 3 70B	1.5	0.0036	Yes
Llama 3 405B	GPT-4 (0125)	2.2	0.0036	Yes
GPT-4o	Llama 3 405B	1.8	0.0036	Yes
Claude 3.5 Sonnet	GPT-4o	0.8	0.065	No

Table 1: All 8 adjacent-rank pairs from the ranked MMLU comparison, Holm-adjusted p-values from the full 36-comparison family. 3 of 8 are not significant.

Three of the eight adjacent-rank gaps – GPT-3.5 Turbo vs. Llama 3 8B, Nemotron-4 340B vs. Llama 3 70B, and GPT-4o vs. Claude 3.5 Sonnet – are not statistically distinguishable at $\alpha = 0.05$

once the correction accounts for the full set of comparisons the ranking implies. Notably, all three land on the identical adjusted p-value (0.065): Holm's step-down procedure enforces monotonicity of adjusted p-values in sorted order, so once a larger adjusted value appears it propagates forward to subsequent, nominally smaller, raw p-values – a real property of the correction, not an artifact, and a concrete illustration of why Holm-adjusted p-values should be read as a corrected significance threshold rather than a precise per-comparison estimate. The full 36-comparison table, and the script that reproduces it end-to-end from the raw accuracy figures, are included with this paper's source.

6 Limitations

The case study reconstructs i.i.d. per-item vectors from aggregate accuracy because true per-item data for these models is not public; this preserves the point estimate and lets us bound significance conservatively (Section 5), but it cannot recover any real correlation structure between items, and a genuine per-item re-analysis – exactly what `evalci`'s `lm-evaluation-harness` and HELM adapters are for – would be preferable wherever per-item logs are available. Separately, the source table itself does not disclose whether every competitor number was reproduced under one evaluation harness or drawn from each vendor's own report; `evalci` computes correctly conditional on the numbers given to it, but cannot correct for evaluation-protocol inconsistency upstream of those numbers. Monte Carlo permutation and bootstrap p-values have finite resolution ($1/(n_{\text{resamples}} + 1)$); very small nominal p-values should use a larger resample count. Wilson and Clopper-Pearson intervals assume independent binary per-item scores; correlated or continuous scores should use `cluster_ci` or the bootstrap methods instead. The power simulation's correlation parameter ρ is a simplified shared-difficulty knob, not a fitted bivariate model, and should be treated as indicative.

7 Availability

`evalci` is released under the MIT license and available via `pip install evalci` (`https://pypi.org/project/evalci/`). Source, issue tracker, and the reproducible case-study scripts are at `https://github.com/Shreyaskc/evalci`. Continuous integration runs the test suite across Python 3.9–3.12 on every push. Tagged releases are archived on Zenodo (concept DOI `https://doi.org/10.5281/zenodo.21201815`).

8 Conclusion

Reviewers increasingly ask evaluation papers for error bars; until now, authors who wanted to answer correctly had to reimplement paired bootstrap and permutation tests by hand, so most did not. `evalci` packages decades-old, individually well-validated statistical methods into the one shape an evaluation actually takes – a per-item table – and validates the implementation against an independent reference rather than only against itself. The case study in Section 5 is not a special case: applied to any public leaderboard, the same one-line `multi_compare` call will show how many of its rank differences are noise. We hope making that call this cheap raises the floor of the field's empirical rigor the way general-purpose scientific-computing libraries have for other parts of the stack.

References

Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.

Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. With little power comes great responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9263–9274, 2020.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with numpy. *Nature*, 585:357–362, 2020.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *International Conference on Learning Representations (ICLR)*, 2021.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

Skipper Seabold and Josef Perktold. statsmodels: Econometric and statistical modeling with python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.